

lawgorithm.

THE COMPLETE CODE GUIDE

How Lawgorithm *actually works.*

Your Rights in Your Language.

Every file in the project, explained in plain words — and the exact journey a contract takes from the moment someone uploads it to the moment they read the result in their own language.

~110

code files

2,273

sections of Indian law

7

languages

What's inside

Read chapters 1–3 first. They give you the whole picture in about ten minutes. The rest is a reference you can dip into, file by file.

1 • The big picture

What Lawgorithm is, in one page, with an everyday analogy.

2 • The exact workflow

The 14 steps a contract goes through, with the file that does each step.

3 • The folder map

Where everything lives.

4 • Backend: the front door

main.py, config.py and the API routes.

5 • Backend: reading the contract

Ingestion, cleaning, privacy, cutting into clauses.

6 • Backend: the AI brain

The agent, its prompts, the summary, the AI providers.

7 • Backend: the law library (RAG)

How the right section of law is found.

8 • Backend: the trust layer

Citation checks, grounding checks, confidence, red flags.

9 • Backend: translation & review

Seven languages, and the human review desk.

10 • Decisions, human review & payments

The clear answer, the expert queue, ₹10 pay-per-use.

11 • The website (frontend)

index.html, style.css, app.js, i18n.js.

12 • Scripts, evaluation, deployment

Building data, training, measuring, going live.

13 • Tests

The 196 automatic checks.

14 • Glossary

Every technical word, explained simply.

15 • Cheat sheet

Commands, numbers and settings at a glance.

The big picture

Lawgorithm reads a rental or job contract and tells you, clause by clause, what it means in plain words, whether it's risky, and which Indian law says so — in your language.

Think of it like a careful junior lawyer at a desk

You hand over a contract. The junior lawyer:

1. **reads** the pages (even a phone photo), and blacks out your phone number, PAN and Aadhaar;
2. **skims the whole thing** once, to know who the parties are, the city, the rent or salary;
3. **cuts it into clauses** and takes them one at a time;
4. for each clause, **walks to the law library** (up to 3 trips), pulls out the relevant sections;
5. **writes a note**: plain meaning, a colour (red / amber / green), and what you should do;
6. a **senior checks the note**: did the junior quote real sections? does the law really say that? how sure are we?
7. the note is **translated** into your language and checked that no number or party got swapped;
8. everything is **shredded within 60 minutes**. Nothing is filed away.

The three big parts

Part	What it is	Where
Website (frontend)	What the user sees: the landing page, upload box, progress screen, results, language picker. Plain HTML + CSS + JavaScript, no framework.	frontend/
Server (backend)	A Python program (FastAPI) that receives the file, runs the whole analysis, and sends results back to the website.	backend/
Knowledge (data & models)	The Indian law library (2,273 sections), a search index over it, and a small trained model that recognises clause types.	data/ , models/ , chroma_db/

Who does the "thinking"?

Three kinds of AI are used, each for what it's best at:

- **Groq (free cloud AI)** — the big language model that reads clauses, decides risk and writes explanations. Two models: `openai/gpt-oss-120b` for the legal reasoning ("agent" role) and `qwen/qwen3.8-27b` for lighter jobs like checking and translating ("light" role).
- **A small trained classifier (MiniLM)** — runs on our own machine, instantly labels a clause as "non-compete", "security deposit", "termination"... (87.7% accurate). It's a hint for the big model.

- **An embedding model (all-MiniLM-L6-v2)** — turns text into numbers so we can find law sections that *mean* the same thing as a clause, not just share words.

The one rule that shapes everything: never bluff

Most AI tools confidently invent laws. Lawgorithm forces every legal claim to point at a section it actually retrieved from the library, has a second AI check that the section really says that, and scores its own confidence from things it can verify. If it isn't sure, it says so and asks for a human check instead of guessing.

Privacy (DPDP Act, 2023)

The user must tick a consent box. Personal numbers are removed before any AI sees the text. Results live only in memory (`STORE_ANALYSES=false`), are erased after 60 minutes, and a "Delete now" button erases them instantly.

The exact workflow

Follow one rental agreement from "upload" to "result". Each step names the file that does the work, so you can open it and look.

1

The user opens the site and picks a language. First visit shows the language picker; the choice is remembered in the browser. All page text comes from the 7-language dictionary.

`frontend/index.html · frontend/i18n.js · frontend/app.js → initLanguage()`

2

They choose Rental or Job, drop a PDF/photo/DOCX, tick consent, press "Decode my contract". The browser sends the file to `POST /analyze-document/start` with the type, the consent flag and the language.

`frontend/app.js → initUpload(), startAnalysis()`

3

The server checks the request and starts a background job. No consent → refused. Not "rental"/"job", or not a PDF/photo/DOCX/TXT → refused. Otherwise the file is saved to a temporary file, a job id is returned immediately, and the work happens on a background thread.

`backend/api/routes_documents.py → start_analysis() · backend/services/jobs.py → submit()`

4

The website waits and shows progress. Every 1.5 seconds it asks `GET /jobs/{id}` "how far are you?" and shows "Reading... Summarising... Clause 3 of 7", showing each clause card as soon as it's done.

`frontend/app.js → poll(), setStage()`

5

Read the file into text. PDFs use their text layer; if that's empty (a scan) or it's a photo, OCR (Tesseract) reads the pixels. DOCX is read paragraph by paragraph. The temporary file is deleted afterwards.

`backend/ingestion/__init__.py · pdf_reader.py · ocr.py · docx_reader.py`

6

Clean the text, then remove personal data. Page numbers, repeated headers and broken lines are fixed. Then emails, phone numbers, PAN, Aadhaar, IFSC, bank/card numbers, GSTIN and passport numbers are replaced with tags like `[PAN]`.

`backend/ingestion/cleaner.py · backend/privacy/redaction.py`

7

Cut the contract into clauses. Headings like "4. NON-COMPETE", "4.1", "CLAUSE 7" or ALL-CAPS titles start a new clause. The preamble (clause 0) is skipped.

`backend/segmentation/clause_segmenter.py`

8

Summarise the whole contract once. One AI call extracts parties, place, term, money, key terms. This is shown to the user and handed to every clause as background, so "24-month lock-in" can be judged against an 11-month lease. The state (e.g. Tamil Nadu) is detected here too.

`backend/agent/summarizer.py · backend/risk/routing_flags.py → detect_states()`

▼ then, for every clause, steps 9–12 repeat ▼

9

Guess the clause type. The trained MiniLM classifier says e.g. "security deposit (0.91)". This gives the AI a hint and a ready-made search query for the law library.

`backend/rag/clause_index.py · clause_classifier.py · clause_taxonomy.py`

10

Lawgorithm looks up the law, then asks the AI once. It searches the library twice — with the clause-type hint's legal search phrase and with the clause's own words — keeps the best 4 sections (skipping other states' rent laws once the state is known) and sends them with the clause in **one request**. This "prefetch" mode uses ~4× fewer AI tokens than letting the AI search in rounds (the older "ReAct" mode, still available with `AGENT_MODE=react`).

`backend/agent/agent.py → run_agent(), _prefetch_law() · prompts.py → SYSTEM_PROMPT_PREFETCH`

11

Each search hits the law library two ways. Meaning-based search (embeddings in Chroma) and keyword search (BM25) both rank the 2,273 sections; the two rankings are merged (reciprocal rank fusion) and the best sections are trimmed to ~1,100 characters each so the request stays small.

`backend/rag/retrieval.py · vector_store.py · keyword_index.py · embeddings.py`

12

The answer is checked, scored, flagged — and turned into a decision. The AI returns JSON: plain explanation, risk colour, legal assessment, recommended action, a **consequence** ("If you don't fix it, ...") and claims each citing source ids. Then:

- **Citation check** — every cited id was really retrieved?
`validation/citation_validator.py`
- **Grounding check** — a second AI call: does that section's text actually support the claim? `validation/claim_validator.py`
- **Confidence score** — 30% retrieval + 30% grounding + 20% citations + 10% consistency + 10% the AI's own opinion. `confidence/engine.py`
- **Status** — grounded (≥ 0.95), partially grounded (≥ 0.80), insufficient grounding, or human review required.
- **Review flags** — conflicting sources, unsure which state's law, unverified source, a law named but not backed, a risky clause marked green without checking.
`risk/routing_flags.py · risk/risk_rules.py`
- **Decision** —  no lawyer needed ·  ask for a change ·  talk to a lawyer ·  needs a human expert check (when the answer isn't confirmed). `risk/decision.py`



13

Results are shown. The page shows the overall decision ("Talk to a lawyer", "Expert check needed"...), the summary, the risk ring (how many red/amber/green), the action plan (every change to ask for, red first — copy or print), and each clause card with its decision — for serious ones: "This clause is important. If you don't fix it, [consequence]. For more detail, talk to a lawyer." — and its law sections. Uncertain clauses can be sent to a human reviewer (chapter 10). If the language isn't English, the explanations are sent to `POST /translate`.

`frontend/app.js → renderResults(), renderRiskRing(), renderActionPlan(), translateResults()`

14

Translation, then erasure. The verified English explanation (never the risk colour, numbers or section ids) is translated, back-translated to English and compared: same meaning? same numbers? same landlord/tenant? If not, the English is shown with a note. After 60 minutes — or when the user presses "Delete now" — the result is gone from memory.

`backend/translation/translator.py · glossary.py · backend/services/jobs.py → delete(), _prune()`

Two ways in, one pipeline

`POST /analyze-document` (waits for the whole result) and `POST /analyze-document/start` (background job, what the website uses) both call the same function: `analyze_file()` in `backend/services/analysis_pipeline.py`. That one function *is* steps 5–12.

The folder map

Grey notes say what each folder is for. Folders you never edit by hand are marked "generated".

```

lawgorithm 2/
├─ backend/           the Python server – all the logic
│   ├── main.py       starts the server, serves the website
│   ├── config.py     every setting in one place
│   ├── api/          the URLs the website calls
│   ├── services/     the pipeline + background jobs
│   ├── ingestion/    PDF / photo / DOCX → text, cleaning
│   ├── privacy/      removes personal numbers
│   ├── segmentation/ text → clauses
│   ├── agent/        the AI researcher (ReAct loop, prompts, summary)
│   ├── llm/          plugs for Groq / Ollama / Gemini
│   ├── rag/          law library search + clause-type classifier
│   ├── validation/   citation + grounding checks
│   ├── confidence/   confidence score + calibration maths
│   ├── risk/         risk colours + review flags
│   ├── translation/  7-language output + safety checks
│   ├── verification/ human review queue
│   └─ database/      optional SQLite storage (off by default)
├─ frontend/          the website: index.html, style.css, app.js, i18n.js
├─ data/
│   ├── statutes/     THE LAW LIBRARY (indian_statutes.json, model_tenancy_act.json)
│   ├── processed/    clause dataset splits (generated)
│   ├── raw/          sample contracts + Indian templates
│   ├── sources/      downloaded Acts + PDFs used to build the library
│   └─ evaluation/    gold_clauses.json – expert answers for testing
├─ models/clause_classifier/ the trained MiniLM clause-type model (generated)
├─ chroma_db/         the search index (generated)
├─ scripts/           one-off jobs: build library, train, deploy...
├─ eval/              measuring accuracy
├─ deploy/space/      files for the live Hugging Face site
├─ tests/             196 automatic checks
├─ requirements.txt   Python packages to install
├─ Dockerfile         recipe to run it in a container
├─ pytest.ini         test settings
├─ .env               YOUR SECRETS (Groq key) – never share, never upload
└─ README.md          long technical readme

```

The front door

These files start the server and decide which URL does what.

backend/main.py71 lines

Creates the FastAPI app, connects the three groups of routes (documents, analysis, verification), and serves the website files (/ , /style.css , /app.js , /i18n.js). Also has /health ("is the server alive?").

Like the reception desk: it doesn't do the work, it points every visitor to the right room.

backend/config.py119 lines

Every setting, read from .env or the environment with a sensible default. The important ones:

Setting	Default	Meaning
LLM_PROVIDER	ollama (groq on the live site)	Which AI to use: groq, ollama or gemini.
GROQ_MODEL_AGENT / _LIGHT	gpt-oss-120b / qwen3.8-27b	Big model for reasoning, smaller model for checks + translation.
AGENT_MAX_STEPS	3	Max library searches per clause.
STORE_ANALYSES	false	DPDP privacy mode: never save contracts.
RESULT_TTL_MINUTES	60	How long a result stays in memory.
TRANSLATION_ENGINE	indictrans2 (llm on live site)	Which translator to try first.
CLAUSE_TYPE_MIN_CONFIDENCE	0.45	Below this the clause-type hint is ignored.
EMBEDDING_MODEL_NAME	all-MiniLM-L6-v2	Model that turns text into numbers for search.
DPDP_ACT_URL	MeitY PDF	Where the privacy link points.

backend/api/routes_documents.py94 lines

The upload URLs. POST /analyze-document (wait for full result), POST /analyze-document/start (start a job, get an id), GET /jobs/{id} (progress + finished clauses), DELETE /jobs/{id} (erase now). Checks consent first (DPDP s.6), saves the upload to a temp file, hands it on.

backend/api/routes_analysis.py88 lines

GET /config tells the website how data is handled (AI service, storage off, 60-minute retention, DPDP link — never secrets). GET /document/{id} re-opens a finished result, DELETE /document/{id} erases it, POST /translate translates explanations.

backend/api/routes_verification.py 122 lines

The human reviewer desk: list clauses waiting for review, record a reviewer's decision (approve / edit / reject / "request more evidence" — which really re-runs the analysis), and list past corrections. Only active when storage is on (off on the live site).

backend/services/analysis_pipeline.py 182 lines

The most important file to understand. `analyze_file()` runs steps 5–12 of the workflow in order: extract → clean → redact → segment → summarise → detect state → agent per clause → overall risk. It reports progress ("clause 3 of 7") through a callback. If the AI is unreachable it stops cleanly with a friendly 503 error; if one clause fails, the others still finish and the failed one is listed.

Like a factory conveyor belt: each station is another file; this file is the belt.

backend/services/jobs.py 142 lines

Background jobs held in memory. One worker thread processes uploads in order (others show "queued"). Keeps each job's progress and results; deletes them after 60 minutes (`_prune`) or immediately on "Delete now".

backend/services/clause_service.py 105 lines

Small helper: turns an analysis result into the shape returned to the website (and into database rows when storage is on), and writes the human-readable reason a clause was sent for review.

Reading the contract

Turning a file into clean, private, numbered pieces of text.

backend/ingestion/__init__.py `extract_document()`

Looks at the file extension and calls the right reader. For PDFs it first tries the text layer and, if there's too little text per page (a scan), falls back to OCR.

backend/ingestion/pdf_reader.py

Reads the PDF's built-in text with `pypdf`. Also decides whether that text is "good enough" (at least 40 characters per page, with real line breaks).

backend/ingestion/ocr.py

Optical Character Recognition: reads text from photos and scanned pages using Tesseract (and poppler to turn PDF pages into images). Gives a clear error if those programs aren't installed.

backend/ingestion/docx_reader.py

Reads Word files paragraph by paragraph, keeping the order.

backend/ingestion/cleaner.py

Tidies the raw text: removes page numbers and headers/footers that repeat on every page, joins sentences that the PDF broke across lines — but carefully keeps clause numbers like "4. NON-COMPETE" on their own line, because the next step depends on them.

backend/privacy/redaction.py

DPDP data minimisation. Finds and replaces: emails, Indian phone numbers, PAN, Aadhaar, IFSC, bank/card numbers, GSTIN, passport numbers. Deliberately doesn't touch money amounts ("Rs 2,50,000") so the analysis stays correct. Returns only *counts* of what it removed, never the values.

Like a black marker over the private bits before anyone else reads the page.

backend/segmentation/clause_segmenter.py

Splits the contract into clauses using patterns common in Indian contracts: "4.", "4.1", "CLAUSE 7", ALL-CAPS headings. Each clause gets an id, a title and its text. Also splits headings that got glued into the middle of a line.

The AI brain

The agent that researches each clause, and the plugs that connect it to an AI service.

backend/agent/agent.py 510 lines — the core

`run_agent()` analyses one clause:

1. gets the clause-type hint (chapter 7);
2. starts a conversation with the AI: system prompt + summary + hint + clause (the clause is put *last*, because models pay most attention to the end);
3. **prefetch (default)**: search the law first, then ask the AI once with the sections included. In the optional **ReAct** mode the AI searches itself, up to 3 rounds;
4. `_finalize()`: cleans up the AI's JSON, then runs every check in chapter 8 and builds the final result.

Special case: a *green* clause with no legal claim (e.g. "salary is ₹50,000") doesn't need grounding — *unless* it's a risk-prone type (eviction, lock-in, penalty, non-compete). Then "no claim" means the AI skipped the check, so it gets the `skipped_legal_check` flag.

If the AI forgets a recommended action, a safe default per colour is used.

backend/agent/prompts.py

The written instructions given to the AI: be simple, cite only what you retrieved, use the risk colours like this, reply in this exact JSON shape.

backend/agent/schemas.py

The exact shape of an answer, checked automatically (Pydantic). Two layers: what the **AI writes** (explanation, risk, assessment, claims, its own confidence) and what **Lawgorithm computes** afterwards (grounding, confidence, status, flags). The AI never fills in the second layer.

backend/agent/tools.py

Defines the one tool the AI can use — `search_legal_reference(query, domain)` — and runs it when asked, remembering every section found so the citation check can later verify the AI only cited what it saw.

backend/agent/summarizer.py

Runs once per contract: asks the AI for parties, location, term, money and key terms in simple English. Tidies up small mistakes in the AI's reply. The summary is context only — it can never be cited as law.

backend/llm/ — the AI plugs

`backend/llm/base.py`

The common "plug shape" every AI service must fit: `start_turn`, `continue_turn`, `simple_completion`. Because the agent only talks to this shape, switching Groq ↔ Ollama ↔ Gemini changes one setting, not the agent. Also defines `LLMUnavailableError` (quota used up, bad key, network down).

`backend/llm/__init__.py`

`get_provider(role)` builds the right plug from `LLM_PROVIDER`. Role "agent" → big model; role "light" → smaller model. With Groq, each model has its own free quota, so splitting doubles the budget.

`backend/llm/openai_compat.py` used on the live site

Talks to Groq (any OpenAI-style API). Handles the free tier: if rate-limited for a few seconds it waits and retries; if the daily cap is hit, it says so plainly. Rescues answers when the model wraps its reply in a fake "json" tool call, and as a last resort retries without tools. Request too large → error (retrieval keeps sections short to avoid this).

`backend/llm/ollama.py`

Runs a model on your own laptop via Ollama (default `qwen2.5:3b`). No key, no quota, but slower (~50 s per clause on 8 GB).

`backend/llm/gemini.py`

The original Google Gemini plug. Kept working, but not used anymore (quota limits).

The law library (RAG)

RAG = "Retrieval-Augmented Generation": before the AI writes, we *retrieve* real law and give it to the AI, so it answers from the book instead of from memory.

Two ways to find a law section — used together

Meaning search (embeddings)

Every section is turned into 384 numbers that capture its meaning. A query like "can landlord keep deposit" lands near sections about deposits even if the words differ. Stored in Chroma.

Keyword search (BM25)

Classic search-engine scoring: rare, exact words count most. Catches legal terms like "restraint of trade" that meaning-search sometimes misses.

Both produce a ranking; they are merged with **reciprocal rank fusion** (score = $1/(60+\text{rank})$ from each list, added up). A section that's high in either list rises to the top.

`data/statutes/indian_statutes.json` the library itself

2,236 sections from 25 Acts — Indian Contract Act, Transfer of Property Act, Registration, Stamp, Specific Relief, Limitation, Arbitration, the four Labour Codes, POSH, Copyright, IT Act, Consumer Protection and more, plus state rent laws (Delhi, Karnataka, Tamil Nadu, West Bengal, Maharashtra, UP) and AP Shops & Establishments. Each section keeps a link to its official source.

`model_tenancy_act.json` adds 3 sections of the Model Tenancy Act, 2021.

`backend/rag/ingest_laws.py`

Loads every JSON file in `data/statutes/` and builds the search index. Run `python -m backend.rag.ingest_laws` after changing the library.

`backend/rag/chunking.py`

One chunk = one full section. Never splits a section in the middle, so its exceptions and provisos stay with it. Gives each chunk a stable id like `ica_1872_section_27`.

`backend/rag/embeddings.py`

Loads the embedding model. If there's no internet to download it, uses a simple offline fallback so the app (and tests) still run.

`backend/rag/vector_store.py`

Everything that touches Chroma (the vector database) lives here — add sections, search by meaning. Swapping to another database would mean rewriting only this file.

`backend/rag/keyword_index.py`

The BM25 keyword index, built in memory from the library in about a second. Section titles count 3× because they're short and precise.

`backend/rag/retrieval.py`

`search_legal_reference_structured()` : runs both searches, fuses them, keeps the top 3. `excerpt()` trims long sections to ~1,500 characters (title + opening + the paragraphs that match the query) so requests fit Groq's free-tier size. `best_retrieval_score()` turns "how close was the best match" into a 0–1 number for the confidence score.

Clause-type hint

`backend/rag/clause_taxonomy.py`

The list of clause types (non-compete, security deposit, lock-in, termination, rent escalation...). The training dataset had 179,000 different heading names; this maps them onto a small set of types. Each type has a domain (rental/employment/general), a search hint for the law library, and a "risk_prone" mark.

`backend/rag/clause_classifier.py`

Loads the fine-tuned MiniLM model from `models/clause_classifier/minilm-l6` and predicts a type with a probability. On the live site it runs on the free GPU.

`backend/rag/clause_index.py`

`classify_clause_type()` : uses the classifier (or, if it isn't there, a "nearest examples vote" over 63k example clauses). Important: these examples are mostly US contracts, so they are **only** used to recognise the type — never cited as law.

The trust layer

Four checks that turn "the AI said so" into "here's how sure we are, and why".

`backend/validation/citation_validator.py` no AI, instant

Does every source id a claim cites appear in the list of sections actually retrieved for this clause? If the AI invents "Section 99 of the Rent Act", this catches it. Also finds Acts/sections named in the text that weren't backed by a retrieved source (→ `unbacked_legal_reference` flag).

`backend/validation/claim_validator.py` a second, independent AI call

A citation can be real but irrelevant. This asks the "light" model: "Here is the claim, here is the cited section's text — does the text support the claim?" Verdicts per claim: supported / partially / unsupported → combined into a grounding score (0–1). If that call fails, a simple word-overlap fallback is used.

`backend/confidence/engine.py`

The final confidence is **not** the AI's opinion of itself:

retrieval 30%

grounding 30%

citations 20%

consist. 10%

AI 10%

Then `determine_status()`: invalid citations → human review; grounding below 0.80 → insufficient grounding; confidence ≥ 0.95 → **grounded**; ≥ 0.80 → **partially grounded**; otherwise human review.

`backend/risk/routing_flags.py` no AI

Named warnings so a reader sees exactly what's uncertain: `conflicting_sources` (e.g. Delhi law and Karnataka law both cited), `jurisdiction_uncertain` (state law used but we don't know the state), `unverified_source`, `unbacked_legal_reference`. Also `detect_states()`: finds states/major cities in the text.

`backend/risk/risk_rules.py`

Decides "send to a human?": yes for red risk, any review flag, or a status that isn't grounded. Also: the whole document's risk = its worst clause.

`backend/confidence/calibration.py`

Maths for checking the confidence is honest ("when we say 0.95, are we right 95% of the time?"): calibration error, Brier score, and a recommended auto-approve threshold from reviewer data.

The three risk colours

Red flag unfair, very one-sided, or may clash with the law — push back. **Heads up** could cost money or freedom — read twice, ask. **Looks fine** standard — just check names, dates, numbers.

Translation & human review

`backend/translation/translator.py` 313 lines

Translates only the **already-verified plain explanation** — never the colour, section numbers, ids or status. Tries engines in order (IndicTrans2 → NLLB → the Groq model; the live site uses Groq). Then three safety checks:

- **Meaning** — translate back to English and compare with the original (similarity score);
- **Numbers** — every amount must survive ("2,50,000", "2.5 lakh" and "250000" count as the same);
- **Roles** — landlord must stay landlord, tenant stay tenant, employer/employee likewise.

If a check fails, the user sees the English with a note instead of a risky translation.

`backend/translation/glossary.py`

Fixed words for the parties in each language (e.g. मकान मालिक for landlord), put in *before* translation so the machine copies them instead of mixing them up.

`backend/translation/indictrans.py` • `indictrans_worker.py`

AI4Bharat's IndicTrans2 model, built for Indian languages. It needs an older library version, so it runs in its own Python environment (`.venvs/indictrans`) as a helper process, talking by JSON lines. Local only.

`backend/translation/nllb.py`

Meta's NLLB-200 translator, a local fallback (non-commercial licence).

Human review & storage (off on the live site)

`backend/verification/verification_service.py`

The review queue: adds flagged clauses, lists pending ones, records a reviewer's decision and keeps an AI-vs-human corrections log for later calibration.

`backend/database/models.py` • `db.py`

SQLite tables (Document, Clause, VerificationRecord, ReviewerCorrection) via SQLAlchemy. Only used when `STORE_ANALYSES=true`. `db.py` creates tables and adds new columns to old databases.

Two review desks

This older desk works only when whole analyses are stored (`STORE_ANALYSES=true`). The live site never stores contracts, so it uses the newer **expert review queue** instead — the person chooses to send just the uncertain clauses, with consent (chapter 10).

Decisions, human review & payments

Turning analysis into a clear answer, bringing a human in when the AI isn't sure, and charging fairly for it.

backend/risk/decision.py

One decision per clause and per contract. A clause is **confirmed** when its claims cite real retrieved sections, the grounding check agrees, and no warning flag fired. Then: green → **no lawyer needed**; confirmed amber → **ask for a change**; confirmed red → **talk to a lawyer**; anything not confirmed → **needs a human expert check**. The contract's headline is the most serious one.

backend/expert_review/ – the human in the loop

expert_review/service.py

The person ticks consent and sends only the uncertain clauses (personal numbers already removed). A **ticket** with a long random id is created — the private link to check the answer. Reviewers see the clause, the AI's analysis and the law; they choose a decision, correct the "If you don't fix it..." phrase and can add a note. Tickets are deleted 7 days after the review and never kept more than 30 days.

Like a sealed envelope: only the clauses you choose go in, only whoever holds the link can open the answer.

expert_review/store.py

Where tickets wait: JSON files locally, or a **private Hugging Face dataset** on the live site (free, survives restarts). Deleting a ticket also wipes the dataset's history, so the data is really gone.

api/routes_expert.py

Send / view / delete a ticket; the reviewer queue; and — owner only — **Manage reviewers**: add someone and get an invite link (their key is stored only as a hash), remove them and the link stops working at once.

backend/payments/ – pay per use

payments/razorpay.py • api/routes_payments.py

₹10 per contract through Razorpay (UPI, cards, netbanking). The page opens Razorpay's secure checkout; the server then checks the payment's signature, asks Razorpay that it's captured, for the right amount and purpose, and marks it **used on Razorpay itself** — so it can't be reused, even after a restart. The **sample contracts are free** (recognised by their exact bytes). If the check fails on our side, the money is **refunded automatically**. The human check is free while reviewers are the team;

`EXPERT_REVIEW_PRICE_PAISE` turns on a price later. With no Razorpay keys, everything is free.

`api/routes_contact.py`

The Contact page form — questions, payment problems and **DPDP grievances**. Messages are stored privately and only the owner reads them on the reviewer page ( Messages). Deleted after 90 days.

Why the human check is free for now

In India only enrolled advocates may practise law (Advocates Act, 1961, s.33). While the reviewers are the Lawgorithm team, the site says so honestly ("a member of the Lawgorithm team") and doesn't charge. When a lawyer or law student reviews, set `REVIEWER_KIND=legal` (and a price if you want).

The website (frontend)

Four files, no framework, no build step. The browser loads them as they are.

frontend/index.html 462 lines — the skeleton

Contains every screen at once; JavaScript shows one at a time ("views"):

- **Home:** the opening (5,86,00,000 pending cases, PENDING files), the upload box, before→after demo, the "spot the red flag" game, how it works, the law library numbers, the privacy promise, FAQ, the finale (PENDING → RESOLVED) and footer;
- **Progress:** the waiting screen with stages;
- **Results:** summary, risk ring, action plan, clause cards, document view;
- **Privacy notice** and **Reviewer desk**;
- two pop-ups: the **language picker** and the review edit dialog.

Any element with `data-i18n="some.key"` gets its text from the dictionary in the chosen language.

frontend/style.css 1,097 lines — the look

Colours are defined once as variables at the top (cream, ink, coral, ...) with a light and a dark set, so the theme toggle just switches sets. Then: nav, buttons, upload card, progress, result cards, the landing sections (opening, case files and stamps, game, marquee, FAQ, finale, footer), phone layout rules, "reduce motion" rules and print rules (so the action plan prints cleanly).

frontend/app.js 967 lines — the behaviour

Organised in labelled sections:

Section	What it does
theme	Light/dark toggle, remembered in the browser.
language	First-visit picker, switching language, re-translating results.
views + routing	Shows the right screen for the address: <code>#/job/...</code> , <code>#/document/...</code> , <code>#/privacy</code> , <code>#/review</code> .
privacy / config	Reads <code>/config</code> and fills in the privacy notice text.
upload	Drag-and-drop, camera photo, sample contracts, consent box, sending the file.
progress	Asks <code>/jobs/{id}</code> every 1.5 s and updates the stage display.
results	Summary, risk ring, action plan (copy/print), clause cards with sources, "Delete now".
translation	Sends explanations to <code>/translate</code> , shows notes when a check failed.
reviewer desk	The review UI (when enabled).
landing page	Fade-in on scroll, the count-up number, the red-flag game.
nav · boot	Menu buttons; start-up order when the page loads.

`h()` is a tiny helper that creates HTML elements; `api()` calls the server; `toast()` shows a small popup message.

frontend/i18n.js 1,929 lines — every word, 7 times

The dictionary: `STRINGS.en`, `.hi`, `.mr`, `.ta`, `.bn`, `.te`, `.kn`, each with the same keys (e.g. `"tagline": "Your Rights in Your Language."`). At the bottom, a tiny runtime: `t(key)` looks up a word, `apply()` fills every `data-i18n` element, `setLang()` switches and remembers the language. To change any text on the site, change it here.

Scripts, evaluation, deployment

Things you run occasionally from the terminal, not part of the live request.

`scripts/` – building the knowledge

`scripts/build_statute_library.py`

Builds the law library: takes the full text of Central Acts (official India Code PDFs via the "indian-legal-acts" dataset) plus state rent-law PDFs, splits them into sections (with OCR for scanned Gazettes), keeps the official source link, and writes `data/statutes/indian_statutes.json`.

`scripts/build_clause_corpus.py`

Reads the huge dataset (`lawgorithm_train.jsonl`), 1.85 million labelled clauses, mostly US SEC contracts), maps headings to clause types, takes a balanced sample, and splits it into train/test files. Also exports the 18 Indian contract templates found in it.

`scripts/train_clause_classifier.py`

Fine-tunes a small BERT-style model on that corpus to recognise clause types. MiniLM-L6 reached 87.7% accuracy and is what's used.

`scripts/deploy_space.py`

Puts the site live: copies backend, frontend, the law library, a freshly built search index and the classifier into `build/space/`, uploads it to the Hugging Face Space, sets the Groq key as a **private secret** (read from `.env`, never printed) and the public settings. Command: `python -m scripts.deploy_space`.

`scripts/add_reviewer.py`

Owner keys for the reviewer page (`python3 -m scripts.add_reviewer Your Name`). Other reviewers are added from the site itself.

`scripts/run_accuracy_eval.py`

Runs the 50 expert-labelled clauses through the live AI and saves the scores to `data/evaluation/latest_metrics.json`. Run it on a fresh day — it uses about one day's free AI allowance.

`scripts/validate_statutes.py`

Lists law sections missing source/provenance information.

`scripts/inspect_dataset.py · build_dataset.py`

Early-phase helpers: inspect raw contracts (formats, personal data patterns) and convert them into processed clause files using the same pipeline as the live API.

`scripts/calibrate_thresholds.py · export_corrections.py`

Use reviewer decisions to check/suggest the auto-approve threshold, and export the corrections log.

`scripts/test_live_agent.py`

Old manual test against the real Gemini API (from before Groq).

`scripts/update_presentation.py`

Edits text in the review presentation (.pptx). Not part of the app.

`eval/ – measuring quality`

`eval/metrics.py · run_evaluation.py`

Run the agent on expert-labelled clauses (`data/evaluation/gold_clauses.json`) and report risk accuracy (target $\geq 90\%$), grounding precision ($\geq 90\%$), hallucination rate ($< 5\%$), abstention accuracy and readability.

`eval/eval_clause_types.py · clause_type_metrics.py`

Measure the clause-type classifier on held-out clauses: accuracy and macro-F1.

`deploy/space/ – the live site on Hugging Face`

`deploy/space/app.py`

The Space's entry point. Free Spaces must be Gradio apps, so this creates a Gradio server and attaches all our routes onto it — the website and API work at "/" exactly as locally.

`deploy/space/gpu_inference.py`

Runs the clause classifier on the free ZeroGPU (a GPU is attached only while a function marked `@spaces.GPU` runs).

`deploy/space/requirements.txt · packages.txt · README.md`

Python packages, system packages (Tesseract, poppler) and the Space's settings card.

`Root files`

`requirements.txt` · `Dockerfile` · `pytest.ini` · `.env.example` · `README.md`

Packages for local use; a container recipe (Python 3.11 + Tesseract, runs on port 8000); test settings; an example settings file (**note:** it still shows the old Gemini settings — the real ones you use are `LLM_PROVIDER=groq` and `GROQ_API_KEY`); and the long technical readme.

Tests

196 automatic checks. Run them all with `python3 -m pytest -q` — they take about 10 seconds and need no internet or API key.

They don't call the real AI. Instead, two fake AIs in `tests/fakes/` play back pre-written answers: **ScriptedProvider** (a fixed sequence of replies, to test the search loop and the 3-search cutoff) and **OracleProvider** (answers from the gold set, to test the evaluation maths). `conftest.py` switches off the clause-type index so tests don't depend on your machine.

Test file	What it proves
<code>test_end_to_end.py</code>	Upload → analysis → result through the real API routes.
<code>test_agent_react_loop.py</code> · <code>test_agent_finalize.py</code>	The search loop, the 3-search cutoff, final answer assembly.
<code>test_citation_validator.py</code> · <code>test_claim_validator.py</code>	Invented citations are caught; grounding verdicts work.
<code>test_confidence_engine.py</code> · <code>test_calibration_and_corrections.py</code>	Confidence weights, status thresholds, calibration maths.
<code>test_routing_flags.py</code> · <code>test_risk_rules.py</code>	Each review flag fires when it should; review routing.
<code>test_privacy_mode.py</code> · <code>test_redaction.py</code>	Nothing is stored, consent is required, erasure works, personal numbers are removed.
<code>test_translation_*.py</code>	Number check (incl. lakh/crore words), role check, fallbacks.
<code>test_openai_compat_provider.py</code> · <code>test_ollama_provider.py</code>	Groq retry/quota handling and failed-generation rescue; Ollama plug.
<code>test_keyword_index.py</code> · <code>test_chunking.py</code> · <code>test_clause_types.py</code>	BM25 finds s.27 for "non-compete"; chunk ids; clause type mapping.
<code>test_cleaner.py</code> · <code>test_segmentation.py</code> · <code>test_summarizer.py</code>	Text cleaning, clause splitting, summary tidy-up.
<code>test_structured_output_validation.py</code> · <code>test_verification_flow.py</code> · <code>test_eval_harness_smoke.py</code>	Answer shape, review desk flow, evaluation harness.

Glossary

Word	Simple meaning
API / endpoint	A URL the website calls to ask the server to do something (e.g. <code>POST /translate</code>).
Backend / frontend	Server program (Python) / what runs in the user's browser (HTML, CSS, JS).
FastAPI	The Python library the server is built with.
LLM	Large Language Model — the AI that reads and writes text (here via Groq).
Provider	Our "plug" for a particular AI service (Groq, Ollama, Gemini).
Agent / ReAct	An AI that can Reason and Act (use a tool), look at the result, and repeat — like a researcher going back to the library.
Tool call	The AI asking us to run a function for it — here, "search the law library for ...".
RAG	Retrieval-Augmented Generation: fetch real documents first, then let the AI answer using them.
Embedding	A list of numbers representing a text's meaning; similar meanings → similar numbers.
Vector store / Chroma	A database that finds the nearest embeddings quickly.
BM25	Classic keyword-search scoring; rare matching words count most.
Reciprocal rank fusion	Merging two ranked lists by adding $1/(60+\text{position})$ from each.
Chunk	One piece of the library put in the index — here, one whole section.
Classifier / fine-tuning	A model that picks a label; fine-tuning = training an existing model further on our own examples.
Citation / grounding	Citation: pointing at a source. Grounding: the source really supports what was said.
Confidence / abstention	How sure we are (0–1) / choosing to say "not sure, needs a human" instead of guessing.
OCR	Reading text out of an image.
Redaction	Blacking out private information.
i18n	"Internationalisation" (18 letters between i and n): making the site multilingual.
Job / polling	Work done in the background / the website asking "done yet?" every 1.5 s.
TTL	"Time to live" — how long something is kept before deletion (60 minutes).

Rate limit / quota	How many AI requests the free plan allows per minute / per day.
Hugging Face Space / ZeroGPU	Free website hosting for AI apps / free GPU time given only to marked functions.
Secret / .env	Private settings like the Groq key — stored in <code>.env</code> locally and as a Space secret online; never in code.
DPDP Act	India's Digital Personal Data Protection Act, 2023 — consent, minimum data, right to erasure.

Cheat sheet

Commands (run inside the project folder)

Command	What it does
<code>uvicorn backend.main:app --reload</code>	Run the site locally → open http://localhost:8000
<code>python3 -m pytest -q</code>	Run all 196 tests.
<code>python -m backend.rag.ingest_laws</code>	Rebuild the search index after editing the law library.
<code>python -m scripts.build_statute_library</code>	Rebuild the law library from sources.
<code>python -m scripts.deploy_space</code>	Publish to the live site (needs <code>hf auth login</code> + Groq key in <code>.env</code>).
<code>python -m scripts.deploy_space --build-only</code>	Build the bundle without uploading.

Where to change things

I want to...	Edit
change any text on the website	<code>frontend/i18n.js</code> (all 7 languages)
change colours / layout	<code>frontend/style.css</code> (variables at the top)
change the pending-cases number	<code>data-count-to</code> in <code>frontend/index.html</code>
add game clauses	<code>GAME</code> list in <code>frontend/app.js</code> + its "why" text in <code>i18n.js</code>
change how the AI explains / its rules	<code>backend/agent/prompts.py</code>
allow more/fewer searches per clause	<code>AGENT_MAX_STEPS</code> setting
change confidence weights / thresholds	<code>backend/confidence/engine.py</code>
add a law / state Act	<code>data/statutes/*.json</code> , then rebuild the index
detect another kind of personal number	<code>backend/privacy/redaction.py</code>
keep results longer/shorter	<code>RESULT_TTL_MINUTES</code>
turn on payments	<code>RAZORPAY_KEY_ID</code> + <code>RAZORPAY_KEY_SECRET</code> in <code>.env</code> , then deploy
change the price	<code>PRICE_ANALYSIS_PAISE</code> (1000 = ₹10), <code>EXPERT_REVIEW_PRICE_PAISE</code>
add / remove a reviewer	Reviewer page →  Manage reviewers (no deploy needed)

Key numbers

Law library	2,273 sections · 26 Acts (incl. Kerala) + Model Tenancy Act
Searches per clause	max 3 · top 3 sections · ~1,500 characters each
Confidence	grounded ≥ 0.95 · partially ≥ 0.80 · grounding floor 0.80
Clause-type model	MiniLM-L6, 87.7% accuracy, hint used if ≥ 0.45
Privacy	consent required · nothing stored · erased after 60 min · "Delete now"
Expert checks	only clauses you send · deleted 7 days after review · max 30 days
Price	₹10 per contract · samples free · automatic refund if a check fails
Groq free tier (per model)	30 requests/min · 1,000/day · 8,000 tokens/min
Live site	https://arnavsingh18-lawgorithm.hf.space

One sentence to remember

A contract is read, cleaned, made private, cut into clauses, summarised once, and then each clause is researched against real Indian law by an AI that is checked, scored and flagged before anything reaches the user — in their own language, and gone within an hour.